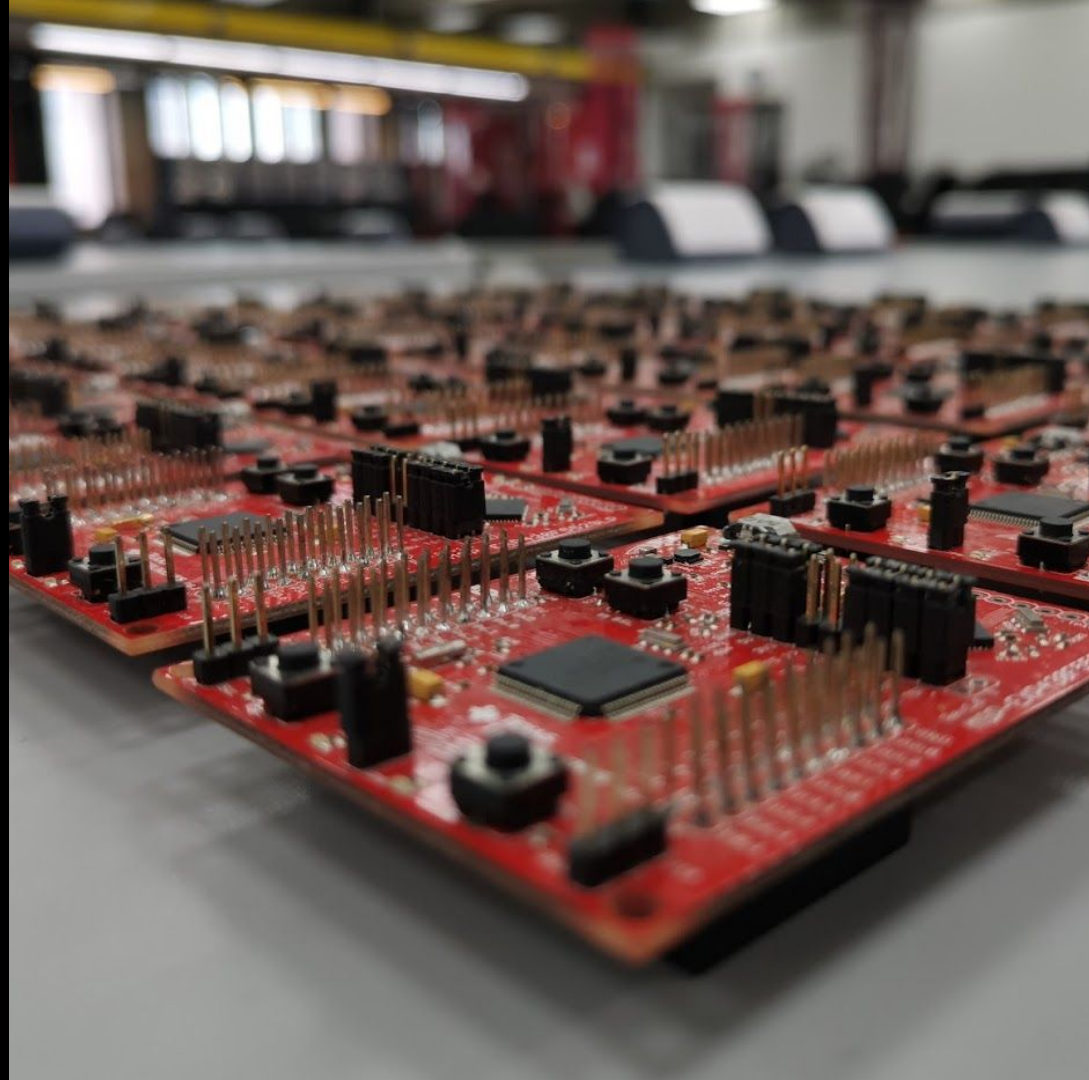
IEE2463 Sistemas Electrónicos Programables

Ayudantía 1
Buenas Prácticas

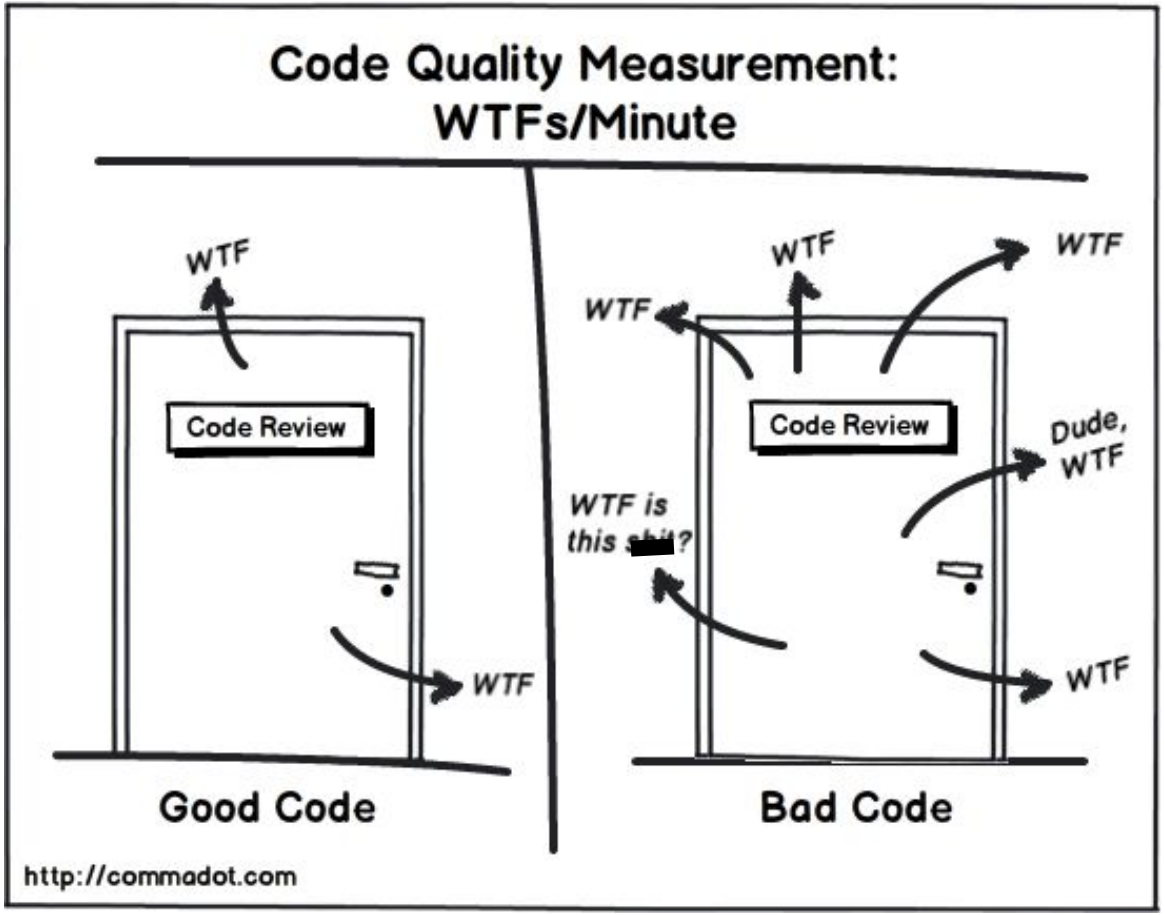
Francisco Fonseca
fffonseca@uc.cl

1er Semestre 2021



Índice

- Importancia de un código ordenado
- Estándares de C y estilos de código
- Preprocesador de C



Fuente: https://commadot.com/wtf-per-minute/

Time heals everything… except bad code.

SSL Bug de Apple

El 15 de febrero del 2015 Apple hizo público iOS 7.0.6, con actualizaciones de seguridad para sus dispositivos móviles en la capa de transporte (SSL). El sistema de seguridad garantizaba una protección contra ataques MITM (Man in the middle).

Después de su lanzamiento un problema salió a la luz, el error se encontraba en el archivo "*sslKeyExchange.c*".

sslKeyExchange.c

```c
    /* more code...*/

    hashOut.data = hashes + SSL_MD5_DIGEST_LEN;
    hashOut.length = SSL_SHA1_DIGEST_LEN;
    if ((err = SSLFreeBuffer(&hashCtx)) != 0)
        goto fail;

    if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto fail;
        goto fail;
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto fail;

    err = sslRawVerify(ctx,
                       ctx->peerPubKey,
                       dataToSign,                /* plaintext */
                       dataToSignLen,            /* plaintext length */
                       signature,
                       signatureLen);
    if(err) {
        sslErrorLog("SSLDecodeSignedServerKeyExchange: sslRawVerify "
                    "returned %d\n", (int)err);
        goto fail;
    }

```

sslKeyExchange.c

```c
    /* more code...*/

    hashOut.data = hashes + SSL_MD5_DIGEST_LEN;
    hashOut.length = SSL_SHA1_DIGEST_LEN;
    if ((err = SSLFreeBuffer(&hashCtx)) != 0)
        goto fail;

    if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom)) != 0)
        goto fail;
    if ((err = SSLHashSHA1.update(&hashCtx, &signedParams)) != 0)
        goto fail;
        goto fail;
    if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
        goto fail;

    err = sslRawVerify(ctx,
                       ctx->peerPubKey,
                       dataToSign,                /* plaintext */
                       dataToSignLen,            /* plaintext length */
                       signature,
                       signatureLen);
    if(err) {
        sslErrorLog("SSLDecodeSignedServerKeyExchange: sslRawVerify "
                    "returned %d\n", (int)err);
        goto fail;
    }

```

Estándares de C

```c
#include<stdio.h>
int main()
{
    int i = 1;
    printf("%d %d %d\n", ++i, i++, i);
    return 0;
}
```

Respuesta en consola:

1) 2 1 3

2) 1 2 3

3) 2 1 3

Estándares de C

```c
#include<stdio.h>
int main()
{
    int i = 1;
    printf("%d %d %d\n", ++i, i++, i);
    return 0;
}
```

Respuesta en consola:

1) 2 1 3

2) 1 2 3

3) 2 1 3

TODAS!!!

Los compiladores de C pueden crear distintos *outputs* dependiendo del estándar usado.

Estilos de Código

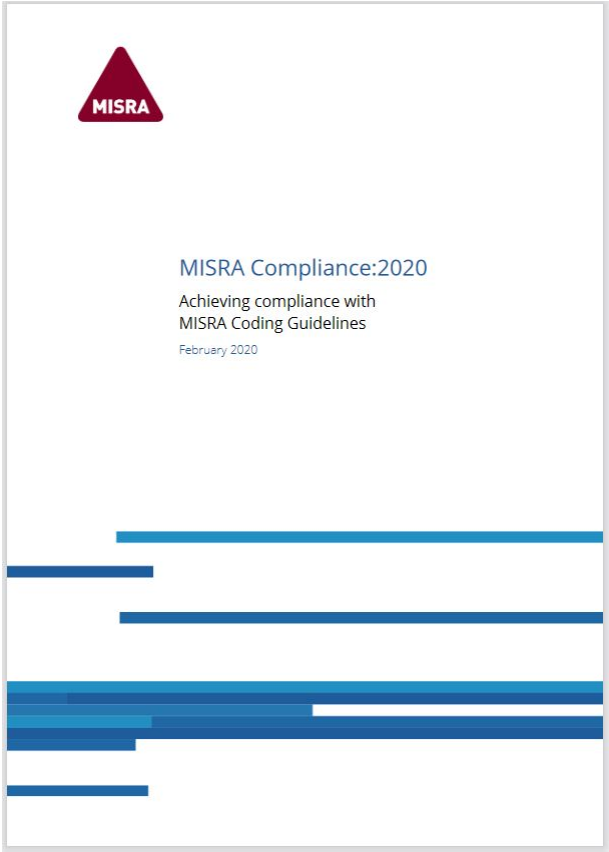
GNOME -> Mira el código alrededor e imítalo

- Líneas de código entre 80 y 120 caracteres.
- Tabulador siempre de 8 espacios.
- En condicionales (If, else), no usar corchetes para declaraciones simples.
- Condiciones demasiado grandes deben estar indentadas, en el mejor de los casos usamos solamente una variable booleana.
- Nuevos bloques deben indentarse.
- Puede comprometerse con los estilos: Linux kernel style o GNU style.
- Entre otros….. revisar: GNOME coding style

Estilos de Código

- Existen otras directrices enfocadas al ámbito profesional como MISRA, el cual es obligatorio para aplicaciones aeroespaciales, vehículos motorizados, etc...

- Estas reglas procuran seguridad, orden y portabilidad en el código.



Preprocesador de C

¿Qué es un preprocesador?

Es un programa que pre-procesa la información de entrada (*source files*) antes de generar la salida (lenguaje de máquina, .exe, .bin, .elf, .out, .hex, etc...).

¿Puedo usarlo OwO?

Claro que sí! De hecho, lo han usado muchas veces al incluir librerías.

```
#include<stdio.h>

int main()
{
    printf("hello world");

    return 0;
}
```

Preprocesador de C

Sin embargo, el preprocesador contiene más funciones que les puede ser útiles a la hora de programar en C. Veamos algunas...